



Escuela de Ingeniería y Tecnologías

Proyecto de Introducción a la Ingeniería

Avances del Proyecto 2

Profesor: Antonio Martinez Torteya.

Angel Gabriel Romero Patena 694550
Esaúl Alejandro Villarreal Ábrego 646239

Este trabajo utilizó el apoyo de la inteligencia artificial de Google Gemini, limitado a la explicación de conceptos y problemas encontrados en el proceso de desarrollo (sin haberse especificado el código exacto, solo el concepto o problema general).

“Doy mi palabra que he realizado esta actividad con integridad académica.”

- Esaúl Alejandro Villarreal Ábrego
- Ángel Gabriel Romero Patena

Introducción

El objetivo de este trabajo es explorar el uso del Arduino para controlar motores DC para después crear una base móvil que sirva para crear un rover para el proyecto final. Se usó como base el circuito integrado L293D, fabricado por Texas Instruments, que permite controlar 2 motores DC de manera simultánea (Texas Instruments, 2016). Las capacidades que serán demostradas en esta tarea son:

- Hacer que los dos motores para que roten en la misma dirección a la misma velocidad.
- Cambiar la dirección hacia la cual ambos motores giran.
- Hacer que uno de los motores gire más lento que el otro, simulando una vuelta si estuvieran en un rover.

Para comenzar, se explicará la teoría relevante previo a mostrar y explicar el código y circuito utilizado. También se realizará un análisis de las mejoras, dificultades, problemas y soluciones encontrados durante el proceso de desarrollo.

Teoría

Puente-H

El Puente-H es un circuito que permite mandar corriente en direcciones opuestas. En el caso de un motor DC, este cambio en la dirección de la corriente implica un cambio en la dirección de rotación (ElectroPeak, 2018). En la **Figura 1**, se puede observar la estructura de un Puente-H.

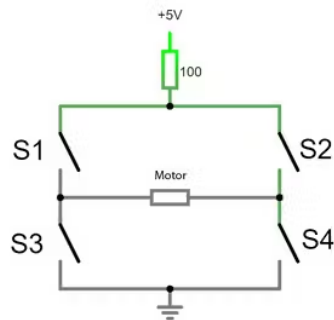


Figura 1: Circuito Puente-H (ElectroPeak, 2018).

Como se puede observar, se hace uso de 4 interruptores. Cuando los interruptores S1 y S4 se cierran, la corriente fluye en una dirección. Cuando S2 y S3 se cierran, la corriente fluye en sentido contrario. Cualquier otra combinación resultaría en un corto circuito (ElectroPeak, 2018).

L293D

Este circuito integrado está compuesto por 4 puentes medio-H, creando 2 puentes-H completos (ElectroPeak, 2018). Al observar la **Figura 2**, se puede identificar las conexiones requeridas para utilizar este dispositivo.

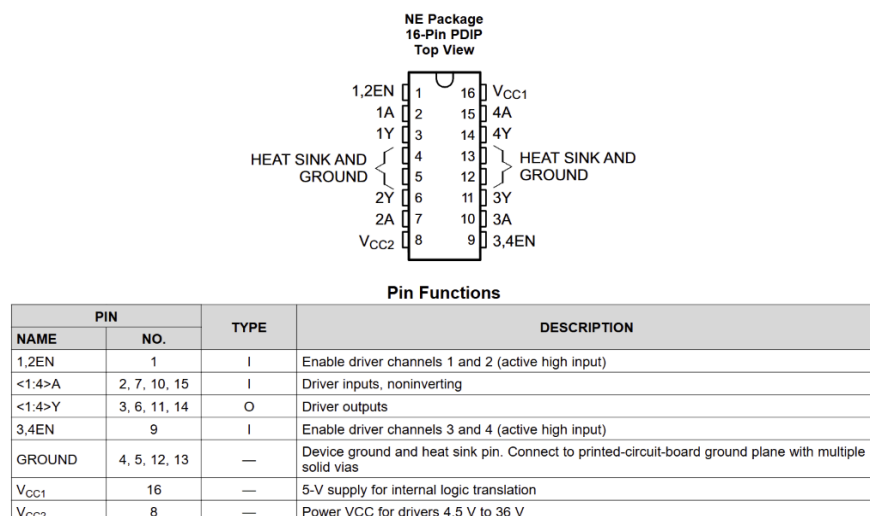


Figura 2: Conexiones del circuito integrado L293D (Texas Instruments, 2016).

Los pines 1 y 9 permiten restringir cuántos y cuales motores puede controlar el L293D y requieren un voltaje alto constante. Los cuatro pares de pines A y Y corresponden a los pares de switches (en este caso transistores) del Puente-H que permiten activar el motor en una dirección. Los pines A, al ser alimentados con 5V, encienden los pines Y, los cuales pueden enviar hasta 600 mA a los motores para que se muevan. Finalmente, cabe notar que existen 4 conexiones a tierra y dos conexiones separadas para el voltaje. La primera conexión es de 5V y alimenta la lógica de los circuitos, mientras que la segunda conexión permite entradas de 4.5V a 36V para alimentar los motores (Texas Instruments, 2016).

Pulse Width Modulation (PWM)

Dado a que la velocidad de rotación de un motor DC es proporcional al voltaje suministrado, se puede aplicar la técnica de PWM para variar ese voltaje utilizando un pin digital del Arduino. En vez de mandar un voltaje variable, se utilizan pulsos con un voltaje de 5V combinado con momentos donde no se suministra ningún voltaje. La proporción de tiempo que dura el pulso de 5V y el tiempo total entre el inicio de un pulso y el siguiente (el periodo) se denomina *ciclo de trabajo*. Dependiendo de este ciclo de trabajo, el voltaje promedio que recibe el motor DC cambia y, por ende, también cambia su velocidad rotacional. Entre mayor sea el ciclo de trabajo (el cual va de 0% a 100%), mayor será el voltaje promedio y la velocidad rotacional (Yousef y Mostafa, 2015).

Una representación de PWM puede verse en la **Figura 4**.

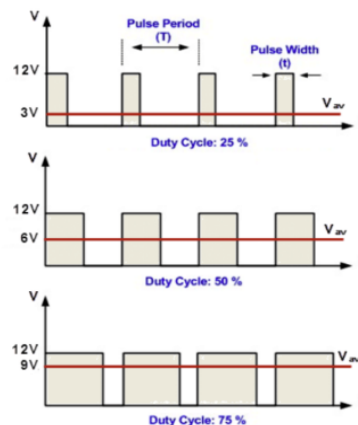


Figura 4: Representación de PWM (Yousef y Mostafa, 2015).

Metodología

Electrónica

En la **Figura 4** se muestra el circuito utilizado como referencia para el circuito utilizado para la tarea.

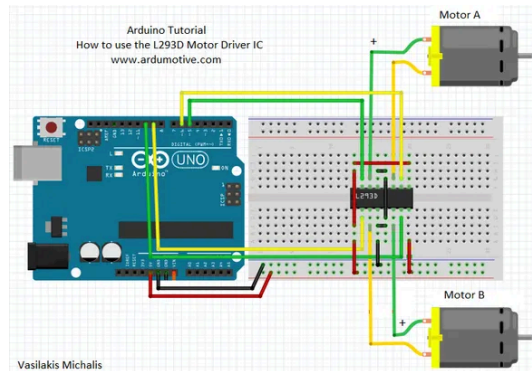


Figura 4: Circuito de referencia utilizado (codebender_cc, 2015).

Al analizar el circuito, se identificó que las conexiones corresponden a aquellas descritas para el L293D en la sección de **Metodología**. Para poder demostrar las diferentes modalidades de motores esperadas (ver **Introducción**), se agregaron 4 botones al circuito de referencia para controlar el comportamiento de los motores DC. El diagrama del circuito final se puede ver en la **Figura 5**.

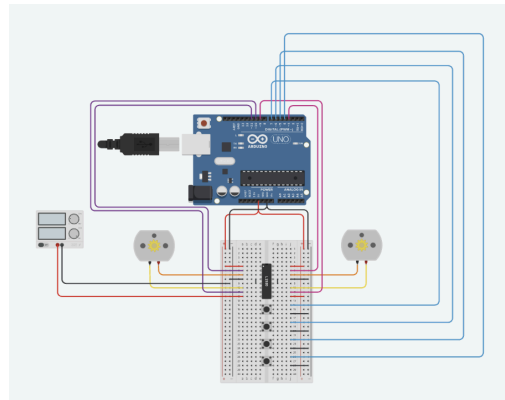


Figura 5: Circuito diseñado en TinkerCad (autoría propia).

Como se puede ver, el voltaje para el circuito proviene del Arduino mientras que el voltaje para los motores proviene de una fuente de poder separada. Esta fuente fue configurada a 6V dado a que este es el voltaje nominal de los motores DC (Wang, 2015).

Los botones usados en el circuito mandan una señal al Arduino al conectar su respectivo pin digital a tierra (0V). Al estar en la configuración de INPUT_PULLUP, esto hace que la lectura que realiza el Arduino vaya de 5V a 0V, permitiendo detectar que un botón ha sido presionado. Esto resulta más conveniente que tener que detectar cuando se recibe una señal de 5V, dado a que interferencia eléctrica del entorno puede causar señales en falso (Arduino, 2024).

Los botones vistos en la **Figura 5**, de arriba hacia abajo, controlan los siguientes estados de los motores.

1. Ambos motores giran hacia la derecha.
2. Ambos motores giran hacia la izquierda.
3. Ambos motores giran hacia la derecha, pero el motor derecho gira a la mitad de la velocidad del motor izquierdo.
4. Ambos motores giran hacia la izquierda, pero el motor izquierdo gira a la mitad de la velocidad del motor derecho.

Para los estados 3 y 4, se hizo uso de PWM para reducir la velocidad de uno de los motores a la mitad al enviarle la mitad del tiempo una salida de 5V y la otra mitad del tiempo una salida de 0V.

Al probar el circuito se identificó que la salida de corriente de la fuente de poder era de 182 mA (velocidad completa para ambos motores) o de 137 mA (velocidad completa en un motor y velocidad parcial en el otro). Esto parecía un error dado a que la corriente de operación de los motores DC (asumiendo que no están cargando nada) es de 90 mA a 120 mA (Wang, 2015).

Sin embargo, al usar un multímetro para medir el voltaje y amperaje de ambos motores, se identificó que la corriente total suministrada por la fuente de poder se dividía entre ambos motores, haciendo que ninguno en realidad superara su rango de operación. En el caso donde se utilizan 182 mA de corriente, cada motor consume ± 90.1 mA. Cuando se utilizan 137 mA, el motor que opera a velocidad completa utiliza ± 90.1 mA, mientras que el que opera a media velocidad usa alrededor de ± 45 mA.

En el caso del voltaje, se observó mediante el multímetro que los motores utilizan ± 5.6 V a velocidad completa y alrededor de ± 2.8 V a media velocidad.

A continuación, en la **Figura 6**, se pueden observar las mediciones de voltaje y amperaje realizadas para los cuatro estados posibles de los motores.

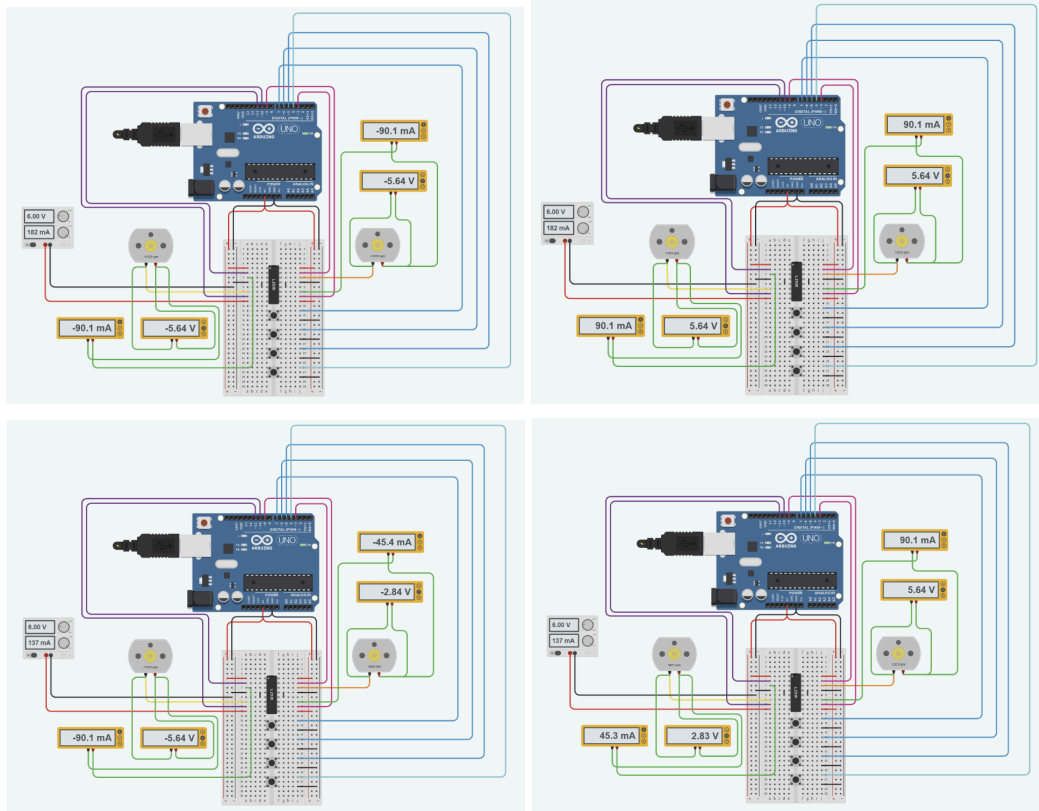


Figura 6: Mediciones de voltaje y amperaje para cada estado posible de los motores (autoría propia).

Programación

Se utilizó una referencia para realizar el código. Parte de esta referencia es visible en la **Figura 7**.

```
L293D_Arduino_Tutorial
by User 31160

L293D_Arduino_Tutorial.ino

10 void setup(){
11
12     //Set pins as outputs
13     pinMode(motorPin1, OUTPUT);
14     pinMode(motorPin2, OUTPUT);
15     pinMode(motorPin3, OUTPUT);
16     pinMode(motorPin4, OUTPUT);
17
18     //Motor Control - Motor A: motorPin1,motorpin2 & Motor B: motorpin3,motorpin4
19
20     //This code will turn Motor A clockwise for 2 sec.
21     digitalWrite(motorPin1, HIGH);
22     digitalWrite(motorPin2, LOW);
23     digitalWrite(motorPin3, LOW);
24     digitalWrite(motorPin4, LOW);
25     delay(2000);
26
27     //This code will turn Motor A counter-clockwise for 2 sec.
28     digitalWrite(motorPin1, LOW);
29     digitalWrite(motorPin2, HIGH);
```

Figura 7: Código de referencia (codebender_cc, 2015).

Como se puede observar, los motores DC son controlados a través de los pines A del L293D, dándoles una señal de LOW o HIGH dependiendo de qué motor va a rotar y en qué dirección lo hará. Con base en en la referencia se realizó el código visto en la **Figura 8**.

```

1 // C++ code
2 //
3 void setup()
4 {
5     //Mientras el botón correspondiente no sea presionado,
6     //el pin leerá una señal de 5V (HIGH).
7     pinMode(4, INPUT_PULLUP);
8     pinMode(5, INPUT_PULLUP);
9     pinMode(6, INPUT_PULLUP);
10    pinMode(7, INPUT_PULLUP);
11
12    //Estas son las conexiones a los pines A del L293D.
13    pinMode(3, OUTPUT);
14    pinMode(9, OUTPUT);
15    pinMode(10, OUTPUT);
16    pinMode(11, OUTPUT);
17 }
18
19 void loop()
20 {
21     //Entre cada presionamiento del botón se apagan los dos motores
22     //con tal de evitar que un estado se sobreponga sobre otro y
23     //cause un posible cortocircuito.
24     digitalWrite(3, LOW);
25     digitalWrite(9, LOW);
26     digitalWrite(10, LOW);
27     digitalWrite(11, LOW);
28
29     //Cada while loop espera que se presione solamente el botón
30     //correspondiente.
31     while(digitalRead(4) == HIGH && digitalRead(5) == HIGH && digitalRead(6) == HIGH && digitalRead(7) == LOW){
32         //Con base en el código de referencia, se mandan señales
33         //de 5V o 0V a los pines correspondientes.
34         digitalWrite(3, HIGH);
35         digitalWrite(9, LOW);
36         digitalWrite(10, HIGH);
37         digitalWrite(11, LOW);
38     }
39
40     while(digitalRead(4) == HIGH && digitalRead(5) == HIGH && digitalRead(6) == LOW && digitalRead(7) == HIGH){
41         digitalWrite(3, LOW);
42         digitalWrite(9, HIGH);
43         digitalWrite(10, LOW);
44         digitalWrite(11, HIGH);
45     }
46
47     while(digitalRead(4) == HIGH && digitalRead(5) == LOW && digitalRead(6) == HIGH && digitalRead(7) == HIGH){
48         //Para los estados donde uno de los motores se mueve a velocidad
49         //media, se utiliza analogWrite para mandar una señal PWM (analogWrite(), 2025)].
50         analogWrite(3, 128);
51         digitalWrite(9, LOW);
52         digitalWrite(10, HIGH);
53         digitalWrite(11, LOW);
54     }
55
56     while(digitalRead(4) == LOW && digitalRead(5) == HIGH && digitalRead(6) == HIGH && digitalRead(7) == HIGH){
57         digitalWrite(3, LOW);
58         digitalWrite(9, HIGH);
59         digitalWrite(10, LOW);
60         analogWrite(11, 128);
61     }
62 }

```

Figura 8: Código utilizado para controlar motores en simulación de TinkerCad (autoría propia).

El código utiliza *whiles loops* para mantener los motores en un estado predeterminado por el tiempo que sea presionado el botón. Las condiciones de cada while loop aseguran que no se puedan presionar múltiples botones a la vez, lo cual podría causar un cortocircuito. A su vez, para generar las señales PWM para controlar la velocidad del motor, se usó la función *analogWrite()*, la cual toma como argumentos el pin digital que manda la señal y el ciclo de trabajo (en un rango de 0 = 0% a 255 = 100%) (*analogWrite()*, 2025). En este caso, se seleccionó un valor de 128 (aproximadamente 50%).

Resultados

El video que demuestra el funcionamiento del circuito se encuentra en el siguiente link: <https://github.com/esaul88/Proyecto-2>

Conclusión

El proyecto representó una oportunidad de aprender nuevos aspectos de la electrónica (el circuito de Puente H) que permiten controlar motores eléctricos DC. Entre los aspectos que se pueden mejorar se encuentra la adición de 2 motores para poder crear un sistema de control para 4 motores, permitiendo crear el control motriz de un rover. A su vez, se podría integrar un L293N, el cual permite controlar 4 motores DC más grandes y potentes.

Progreso de Proyecto Final

Como parte del proyecto final se busca controlar un rover, por lo cual el siguiente código ha sido diseñado con el objetivo de dirigir el rover a lo largo de un terreno buscando la región con menos obstáculos.

```
//(Margolis, 2026)
#include <Servo.h>
```

```
Servo servo1; //definir objeto conectado al servo
```

```
#define trigger 2 //pin para mandar pulsos ultrasónicos.
#define echo 3 //pin para recibir información de sensor ultrasónico.
#define ENA 9 // pin para controlar los 2 motores de la izquierda//
#define ENA2 10
#define EN 11 //pin para habilitar motores//
#define ENB 12 //pin para controlar los 2 motores de la derecha//
#define ENB2 13
int deteccion1; //cantidad de obstaculos que detecta el rover en el area de 0 - 60 grados//
int deteccion2; //cantidad de obstaculos que detecta el rover en el area de 60 - 120 grados//
int deteccion3; //cantidad de obstaculos que detecta el rover en el area de 120 - 180 grados
int rotacionentera; //cantidad de tiempo que tarda el rover para dar la vuelta entera//
int rotaciond; //cantidad de tiempo que tarda el rover en dar una vuelta hacia la izquierda//
int rotacioni; //cantidad de tiempo que tarda el rover en dar una vuelta hacia la derecha//
int avance; //tiempo que avanzara el rover//
```

```
int ultrasonic();
int movement();
int rotar();
int region();
int radar();
int del();
void izquierda();
void derecha();
void giro();
void avanzar();
```

```

void setup(){
  Serial.begin(9600);
  pinMode(ENA, OUTPUT);
  pinMode(ENA2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(ENB2, OUTPUT);
  pinMode(EN, OUTPUT);
  pinMode(trigger, OUTPUT);
  pinMode(echo, INPUT);
  servo1.attach(8);
  digitalWrite(trigger, LOW);
}

```

```

void loop(){ //Funcion principal, radar evalua todo el espacio de 0 - 180 grados, region
//selecciona una parte con menos obstaculos, y se toma la decision de avanzar o rotar
  radar();
  int estado = region(deteccion1, deteccion2, deteccion3);
  if(estado == 1){
    movement();
  }
  else{
    rotar(estado);
  }
  deteccion1 = 0;
  deteccion2 = 0;
  deteccion3 = 0;
}

```

```

//(NaylampMechatronics.com, nd)
int ultrasonic(){ //Funcion que mide las distancias con los obstaculos, si encuentra
//obstaculos regresa 1
  digitalWrite(trigger, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigger, LOW);
  unsigned long duracion = pulseIn(echo, HIGH, 2500);
  float distancia = (duracion * 0.0343) / 2.0;
  if(distancia >= 25.0){
    Serial.println("Seguro para avanzar");
    return 0;
  }
  if(distancia < 25.0 && distancia > 0){
    Serial.print("Objeto detectado a:");
    Serial.print(distancia);
    Serial.println(" cm");
    return 1;
  }
}
}

```

```
int movement(){ //Funcion para mover el rover hacia adelante solamente//
    digitalWrite(EN, HIGH);
    avanzar();
    digitalWrite(EN, LOW);
}
```

```
int rotar(int i){ //Funcion para rotar el rover hacia la derecha, izquierda o media vuelta//
    digitalWrite(EN, HIGH);
    if(i == 0){
        izquierda();
    }
```

```
    if(i == 2){
        derecha();
    }
    if(i == 3){
        giro();
    }
```

```
int estado = ultrasonic();
if(estado == 1){
    digitalWrite(EN, LOW);
    return 1;
}
else{
    digitalWrite(EN, LOW);
    return 0;
}
}
```

```
int region(int a, int b, int c){ //Funcion que evalua cual de las 3 regiones tiene menos
obstaculos//
    if(b == 0){
        return 1;
    }
    if(a == 0){
        return 0;
    }
    if(c == 0){
        return 2;
    }
    return 3;
}
```

```

void radar(){ //Funcion que mide y cuenta cuantos obstaculos hay en cada region//
  for(int x = 0; x < 60; x = x + 10){
    servo1.write(x);
    delay(100);
    int valor = ultrasonic();
    deteccion1 = deteccion1 + valor;
  }
  for(int x = 60; x < 120; x = x + 10){
    servo1.write(x);
    delay(100);
    int valor = ultrasonic();
    deteccion2 = deteccion2 + valor;
  }
  for(int x = 120; x < 180; x = x + 10){
    servo1.write(x);
    delay(100);
    int valor = ultrasonic();
    deteccion3 = deteccion3 + valor;
  }
  for(int x = 180; x >= 90; x = x - 1){
    servo1.write(x);
    delay(5);
  }
}

int del(unsigned long t){
//Función que suprime delay, en vez el rover no se bloquea y puede seguir detectando
//obstáculos
  unsigned long start = millis();
  while(millis() - start < t){
    int detect = ultrasonic();
    if(detect == 1){
      digitalWrite(EN, LOW);
      return 1;
    }
    delay(20);
  }
  return 0;
}

```

```
//(handsontec.com, nd)
```

```
void izquierda(){  
    digitalWrite(ENA, LOW);  
    digitalWrite(ENA2, HIGH);  
    digitalWrite(ENB, HIGH);  
    digitalWrite(ENB2, LOW);  
    del(rotacioni);  
}
```

```
void derecha(){  
    digitalWrite(ENA, HIGH);  
    digitalWrite(ENA2, LOW);  
    digitalWrite(ENB, LOW);  
    digitalWrite(ENB2, HIGH);  
    del(rotaciond);  
}
```

```
void giro(){  
    digitalWrite(ENA, HIGH);  
    digitalWrite(ENA2, LOW);  
    digitalWrite(ENB, LOW);  
    digitalWrite(ENB2, HIGH);  
    del(rotacionentera);  
}
```

```
void avanzar(){  
    digitalWrite(ENA, HIGH);  
    digitalWrite(ENA2, LOW);  
    digitalWrite(ENB, HIGH);  
    digitalWrite(ENB2, LOW);  
    del(avance);  
}
```

Referencias

analogWrite(). (2025, 9 de mayo). ArduinoDocs.

<https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWrite/>

codebender_cc. (2015, 3 de agosto). *How to use the L293D motor driver - arduino tutorial*. Autodesk Instructables.

<https://www.instructables.com/How-to-use-the-L293D-Motor-Driver-Arduino-Tutorial/>

ElectroPeak. (2018, 26 de agosto). *L293D: theory, diagram, simulation & pinout*. hackster.io.

<https://www.hackster.io/electropeak/l293d-theory-diagram-simulation-pinout-4319cd>

INPUT | INPUT_PULLUP | OUTPUT. (2024, 15 de mayo). ArduinoDocs.

<https://docs.arduino.cc/language-reference/en/variables/constants/inputOutputPullup/>

L298N dual h-bridge motor driver. (n.d.).

<https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>

Texas Instruments. (2016, enero). *L298x quadruple half-h drivers*.

https://www.ti.com/lit/ds/symlink/l293d.pdf?ts=1790440415844&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FL293D%253Futm_source%253Dgoogle%2526utm_medium%253Dcpc%2526utm_campaign%253Dasc-null-null-GPN_EN-cpc-pf-google-ww_en_cons%2526utm_content%253DL293D%2526ds_k%253DL293D+Datasheet%2526DCM%253Dyes%2526gclid%253Daw.ds%2526gad_source%253D1%2526gad_campaignid%253D14388345080%2526gclid%253DCj0KCQjw2t3VBhD8ARIsAK0F6qxaA-teNp7cVMdnh-LCD5hbSS0ZX1iQmrt0S2mdSsp4a_RStQWDcdcaAqfaEALw_wcB#page=1.13

Margolis, M. (2026, 26 de septiembre). *Servo*. ArduinoDocs.

<https://docs.arduino.cc/libraries/servo/>

Tutorial de Arduino y Sensor Ultrasónico HC-SR04. (n.d.). Naylamp Mechatronics - Perú.

https://naylampmechatronics.com/blog/10_tutorial-de-arduino-y-sensor-ultrasonico-hc-sr04.html

Wang, K. (2015, 15 de junio). *TFK-280SA-22125*. Arduino.

<https://www.arduino.cc/documents/datasheets/DCmotor.PDF>

Yousef, A. Y., y Mostafa, M.H. (2015). Bidirectional Speed Control of DC Motor Based on Pulse Width Modulation using Microcontroller. *IJSRST*, 1(5), 52-60.

https://www.researchgate.net/publication/284409560_Bidirectional_Speed_Control_of_DC_Motor_Based_on_Pulse_Width_Modulation_using_Microcontroller